

TradeHub

Deploying the 52-Week High Scanner Backend

A step-by-step, illustrated guide to setting up the Supabase + GitHub Actions pipeline that scans the full NSE daily.

Short answer to “do I have to do this on my own computer?”

Yes, for the one-time setup (and any time you change the backend code) — the Supabase CLI commands (login, deploy, etc.) must run from a real terminal, which GitHub Pages cannot provide since it only serves static files. But that setup is one-time. After it's done, the **daily scan runs automatically on GitHub's own servers** via a scheduled GitHub Actions workflow — your computer does not need to be on, and you never touch a terminal again unless you change the code.

1. How the pieces fit together

Four things are involved: your computer (used once, for setup), GitHub (hosts the site and runs the daily trigger), Supabase (stores results and runs the scanning code), and NSE's own website (the source of truth for new 52-week highs). The diagram below shows how data flows between them.

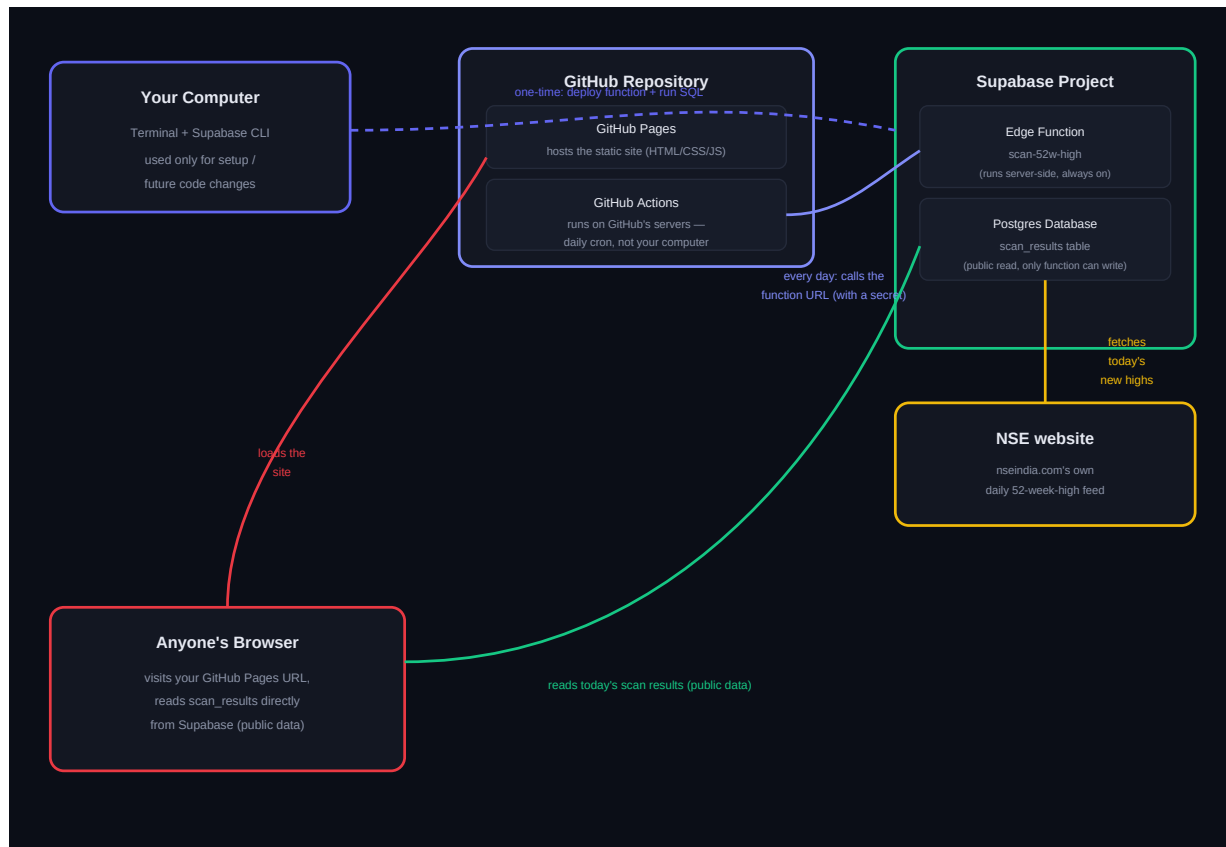


Figure 1 — Data flow between your computer, GitHub, Supabase, NSE, and site visitors.

2. Who runs what, and when

This is the part that answers “do I have to do this on my system” directly: only the left-hand column below happens on your computer, and only once.

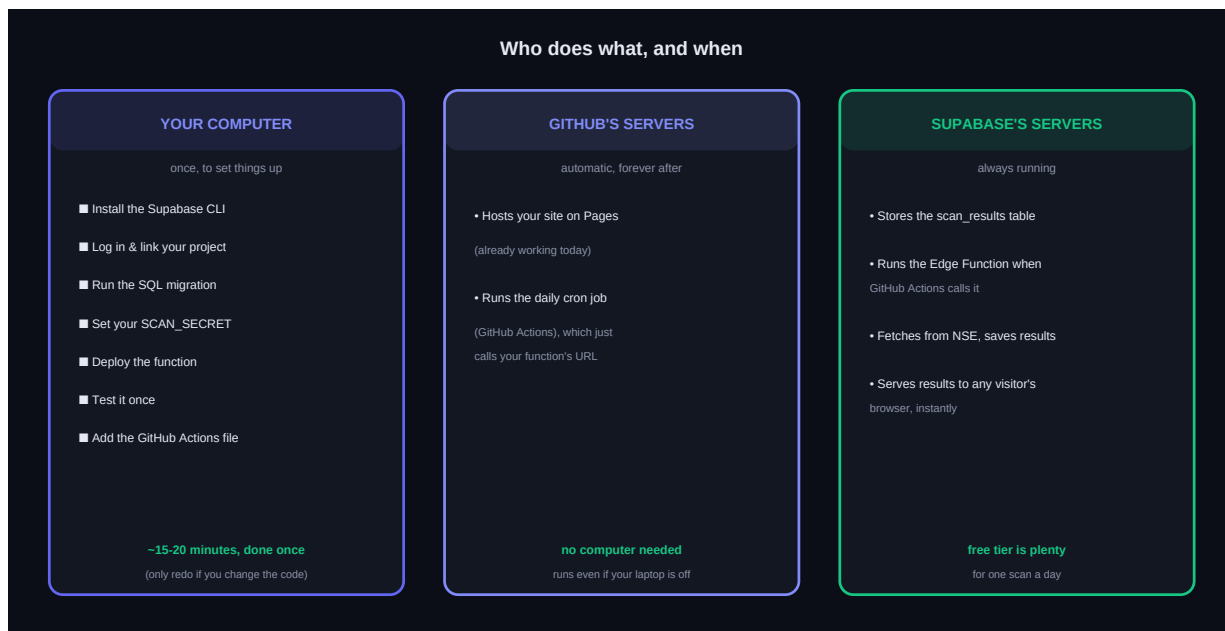


Figure 2 — Setup happens on your computer once; the daily run happens on GitHub and Supabase's servers forever after.

3. Before you start

You'll need:

- Node.js installed on your computer (the Supabase CLI installs via npm).
- Your Supabase project already created — you have this: project ref `hhvtl_zrvvarytohm_xqld`.
- The `supabase-scan52w/` folder from your TradeHub repo, checked out locally (or downloaded).
- Admin access to your GitHub repo (to add a secret and a workflow file).

1 Install the Supabase CLI

```
# one-time install, run in any terminal on your computer
$ npm install -g supabase
$ supabase --version
```

2 Log in and link your project

```
$ supabase login
# opens a browser window to authorize the CLI -- approve it, then return to
the terminal
$ cd path/to/supabase-scan52w
$ supabase link --project-ref hhvtl_zrvvarytohm_xqld
```

“link” just tells the CLI which of your Supabase projects to talk to. It doesn't change anything yet.

3 Create the scan_results table

Open the Supabase dashboard in your browser, go to SQL Editor → New query, paste the full contents of `supabase-scan52w/migrations/0001_scan52w_schema.sql`, and click Run. This creates the table the scanner writes into and the app reads from.

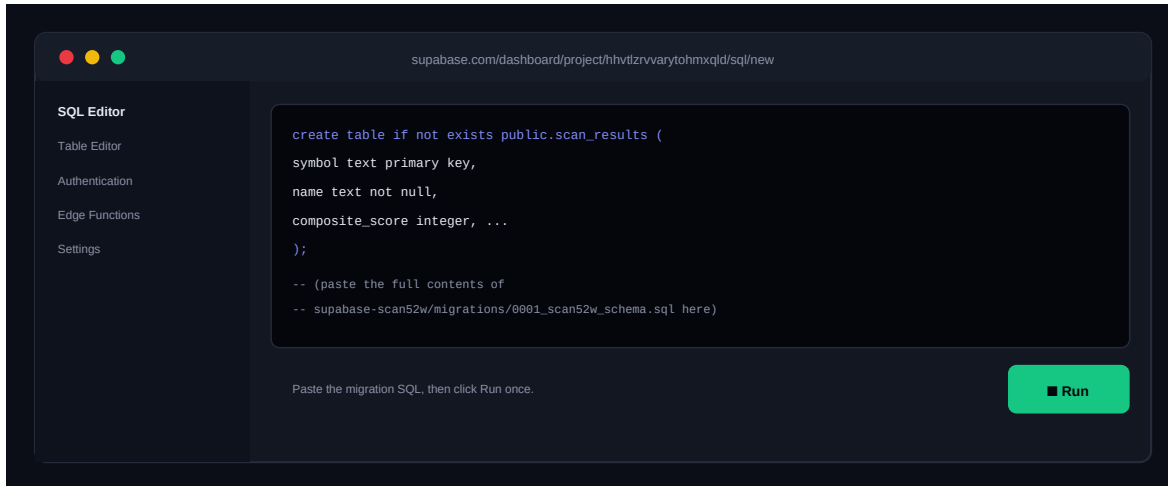


Figure 3 — Illustrative mockup of the Supabase SQL Editor (not a literal screenshot).

4 Set the scan secret

```
# pick any long random string -- this protects your function from being
triggered by strangers
$ supabase secrets set SCAN_SECRET=your-own-long-random-string-here
```

5 Deploy the Edge Function

```
$ supabase functions deploy scan-52w-high --no-verify-jwt
```

This uploads `functions/scan-52w-high/index.ts` to Supabase's servers, where it will live and run from now on — you don't redeploy this again unless you edit the code.

6

Test it once

Open this URL in your browser (with your real project ref and secret filled in) to trigger one scan manually and confirm it works before automating it:

```
$ https://hhvtlrvvarytohmqlld.supabase.co/functions/v1/scan-52w-high?secret=your-secret-here
```

A successful response looks like `{"ok":true,"scan":{"scanned":123,...}}`. Check the `scan_results` table in Table Editor to confirm rows appeared.

7

Automate the daily run with GitHub Actions

This is the step that moves the daily trigger off your computer entirely. Add your function URL and secret as GitHub repo secrets:

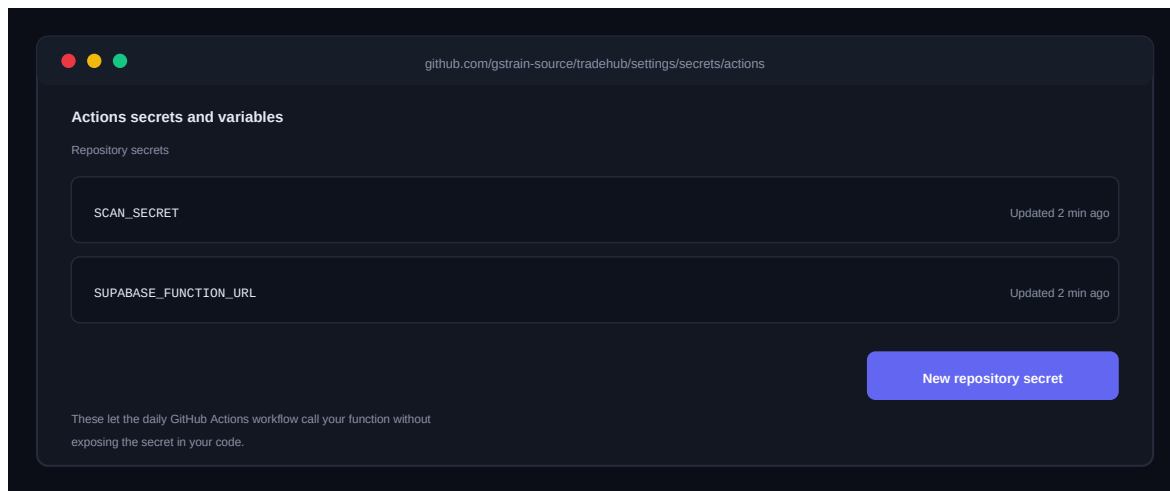


Figure 4 — GitHub → Settings → Secrets and variables → Actions (illustrative).

Then commit a workflow file at `.github/workflows/scan-52w.yml`:

```
$ name: Daily 52-week-high scan
$ on:
$ schedule:
$ - cron: '30 3 * * *' # ~9:00am IST daily; adjust as needed
$ workflow_dispatch: {} # lets you also trigger it manually from the Actions
  tab
$ jobs:
$ scan:
$ runs-on: ubuntu-latest
$ steps:
$ - run: curl "${{ secrets.SUPABASE_FUNCTION_URL }}?secret=${{
  secrets.SCAN_SECRET }}"
```

Once this file is pushed, GitHub runs it every day on its own servers — no CLI, no terminal, no computer of yours involved.

Visit <https://gstrain-source.github.io/tradehub/>, open the 52-Week High page, and switch to the “Full NSE” tab. It reads directly from the `scan_results` table, so once a scan has run you should see today's NSE-confirmed new highs there.

Troubleshooting

Symptom	Likely cause / fix
Full NSE tab shows “not configured”	Check <code>auth-config.js</code> has your real Supabase URL and anon key (not a placeholder).
Full NSE tab loads but is empty	The daily function hasn't run yet, or ran and found 0 new highs that day. Trigger Step 6's URL manually to check.
Test URL in Step 6 returns an error	Confirm the secret in the URL matches what you set with <code>supabase secrets set</code> , and that the function deployed without errors.
GitHub Actions run shows a failure	Open the failed run's log in the Actions tab — usually a missing/mismatched repo secret name.
Need to update the scanning logic later	Edit <code>functions/scan-52w-high/index.ts</code> locally, then re-run just Step 5 (deploy) — the rest stays as-is.

Full command reference and known limitations are also documented in `supabase-scan52w/README.md` in the repo.